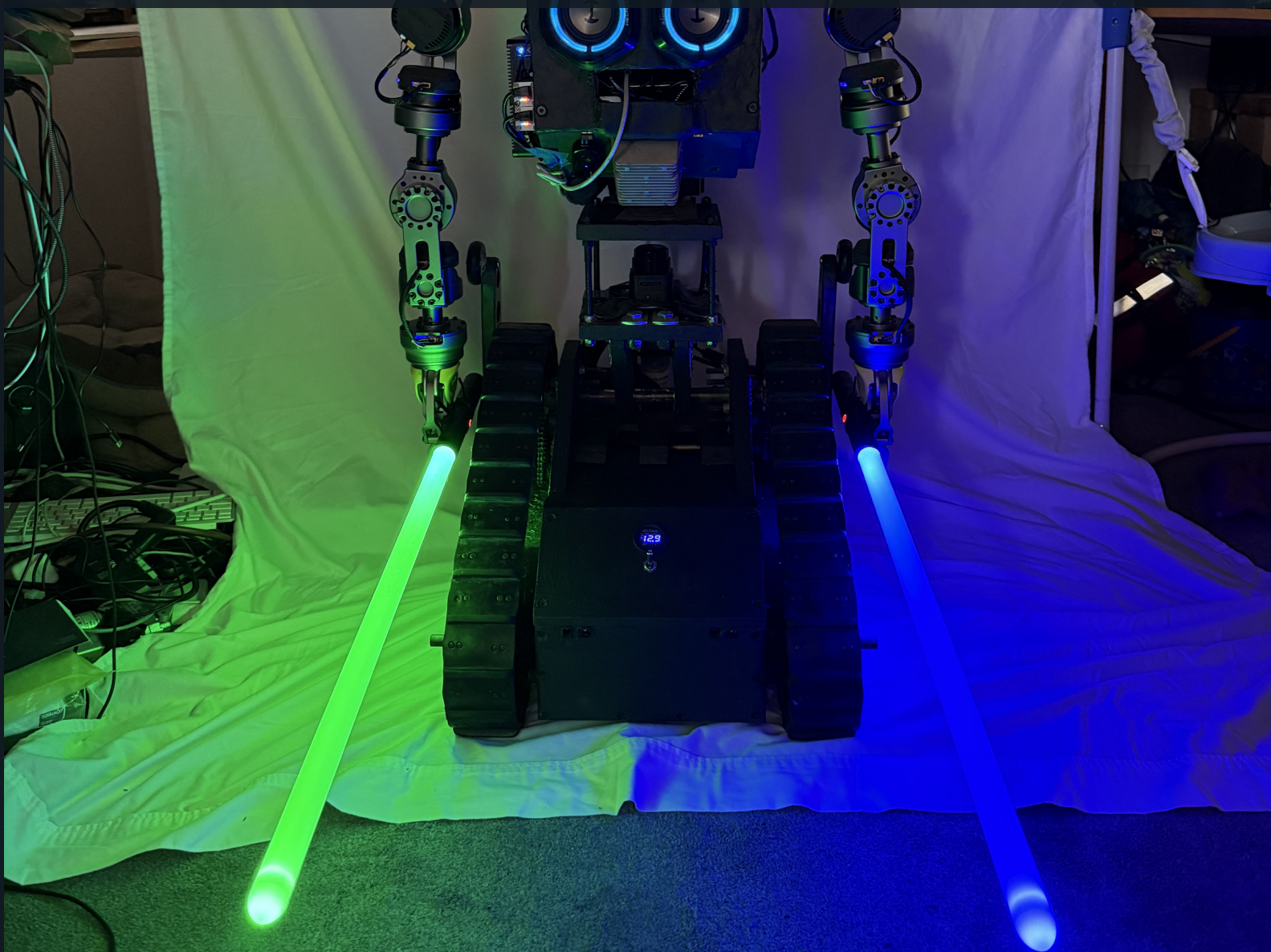

BUILDING R.O.B. - VOLUME 5

AI, ROBOTICS, AND CODEX

The accelerated evolution of ROBController and Cerebro



A SOURCE-BASED FIELD GUIDE FOR ADVANCED MAKERS

Rodolfo Aramayo / Orbitus Robotics - First Maker Faire Edition

A human builder remains responsible

Use AI as an engineering collaborator while preserving evidence, review, and physical safety.

This volume reconstructs the software history of ROBController and Cerebro and turns that history into a practical workflow for working with Codex and OpenAI. It distinguishes repository evidence from interpretation, shows how mature control and perception systems evolved, and keeps authority for physical motion with deterministic software and an accountable operator.

The manuscript reflects the repositories inspected on 10 August 2026. Product behavior and guidance can change; verify current OpenAI guidance against official documentation before publication or training.

Copyright © 2026 Rodolfo Aramayo / Orbitus Robotics. Photographs are from the private ROB build archive unless otherwise noted. The generated frontispiece is original artwork derived from a ROB reference photograph. This independent educational project is not affiliated with or endorsed by any film studio or franchise owner. Product and company names remain the property of their respective owners.

First evidence-based draft: 2 August 2026. Build details marked **Builder input needed** are deliberate placeholders for the builder to complete.

AI OUTPUT IS NOT MOTION AUTHORITY

Generated code, plans, and tool calls require human review. Test protocol and control changes with fixtures, simulators, lifted treads, strict limits, a spotter, and an independently verified physical emergency stop before operating powered hardware.

Contents

- 1 Building R.O.B. 1**
 - 1.1 AI, Robotics, and the Codex-Accelerated Evolution of ROBController and Cerebro 1
 - 1.2 Preface: one builder, many keyboards 1
 - 1.3 1. The robot is a distributed system 1
 - 1.4 2. Era one: handmade foundations, 2022-2023 2
 - 1.4.1 What Codex should learn from this era 2
 - 1.5 3. Era two: the solo workshop, August-September 2025 2
 - 1.5.1 Turning workshop knowledge into specifications 3
 - 1.6 4. The acceleration hinge: August 1, 2026 3
 - 1.6.1 Why this resembles AI-assisted engineering 4
 - 1.7 5. Rebuilding the secure control plane with Codex 4
 - 1.7.1 5.1 Start with a contract 4
 - 1.7.2 5.2 Implement the shared wire format first 4
 - 1.7.3 5.3 Add transport without weakening authentication 5
 - 1.8 6. Operator authority, Watch control, and autonomy 5
 - 1.9 7. Vision, depth, and video as bounded pipelines 6
 - 1.10 8. AI inside Cerebro: proposals, tools, and speech 6
 - 1.10.1 A safe OpenAI integration task for Codex 7
 - 1.11 9. Local improvisation and stage shows 7
 - 1.12 10. Dependencies are part of robot safety 8
 - 1.13 11. A repeatable Codex workflow for these repositories 8
 - 1.13.1 Step 1: protect the worktree 8
 - 1.13.2 Step 2: state outcome and invariants 8
 - 1.13.3 Step 3: ask for read-only diagnosis 9
 - 1.13.4 Step 4: divide at stable boundaries 9
 - 1.13.5 Step 5: make Codex edit and verify 9
 - 1.13.6 Step 6: inspect the diff yourself 9
 - 1.13.7 Step 7: validate in layers 9
 - 1.13.8 Step 8: commit the reasoning 9
 - 1.14 12. Prompt recipes for the actual change families 10
 - 1.14.1 Cross-repository protocol change 10
 - 1.14.2 Concurrency bug 10

- 1.14.3 Camera crash 10
 - 1.14.4 New AI tool 10
 - 1.14.5 Arm feature 10
 - 1.14.6 Documentation from code 10
- 1.15 13. Reviewing AI-generated robot code 10
 - 1.15.1 Safety 10
 - 1.15.2 Security 11
 - 1.15.3 Concurrency 11
 - 1.15.4 Compatibility 11
 - 1.15.5 Evidence 11
- 1.16 14. What remains intentionally unfinished 11
- 1.17 Epilogue: authorship after acceleration 12
- 1.18 Sources and further reading 12
 - 1.18.1 Repository sources 12
 - 1.18.2 Official OpenAI documentation 13

Building R.O.B.

AI, Robotics, and the Codex-Accelerated Evolution of ROBController and Cerebro

A source-based field guide

First edition, August 10, 2026

Preface: one builder, many keyboards

R.O.B. did not begin as an “AI-written robot.” It began the normal way: one person, several computers, physical hardware, partial ideas, experiments that worked once, experiments that broke connectors, and commits made before a fragile discovery disappeared.

The Git record uses three author-name spellings—Orbitus, RudyAramayo, and Rudy Aramayo—but the same orbitus@orbitusrobotics.com email. That is consistent with one builder committing from differently configured computers. It is not evidence of three developers. Nor can Git prove which lines were suggested by an AI. Commit metadata records the person who accepted and committed a change, not every tool involved in producing it.

Still, the history has a visible hinge. Through September 2025, commits read like a lab notebook: tune a servo, repair a port, checkpoint an animation, avoid sending commands too quickly. Beginning August 1, 2026, the repositories gain coordinated transport protocols, certificate pinning, Keychain-backed pairing, formal message schemas, fixture tests, failure boundaries, and long operator documents. ROBController’s first large 2026 change adds roughly 5,345 lines and Cerebro’s adds roughly 16,478. That is the point at which the work *looks AI-accelerated*.

This book treats August 1, 2026 as the start of the Codex-accelerated era. That is an interpretation based on scale and engineering form, not an authorship claim. Rudy remains the author and operator: the person choosing the goal, checking the diff, testing the hardware, and assuming responsibility for what the robot does.

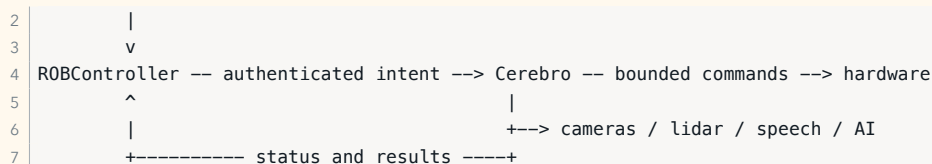
1. The robot is a distributed system

R.O.B. is best understood as two cooperating products.

ROBController is the operator edge. It runs on Apple mobile devices, accepts human intent, shows state, relays Watch commands, and acts as the approval console. Cerebro is the robot-side macOS application. It owns perception, speech, AI sessions, serial hardware, Python helpers, arm scripts, autonomy, stage shows, and the server side of the control link.

The important flow is:

1 human / Apple Watch



This architecture implies a rule that should govern every future change:

Generative AI may propose intent; deterministic software and an accountable operator decide whether that intent can become physical motion.

Robot code has consequences that ordinary application code does not. A UI bug is inconvenient. A stale nonzero tread command, a wrong joint sign, or an unbounded action queue can injure someone or damage hardware. The mature code therefore does more than add intelligence. It constrains intelligence.

2. Era one: handmade foundations, 2022–2023

ROBController's 2022 root commit established the controller around AutoNet and ROBONet after earlier ML-model experiments were removed. It mixed Objective-C, Objective-C++, Swift, C, and C++, plus Apple's AurioTouch sample-derived audio machinery. This was a practical choice: keep the old hardware-facing code and add modern Apple code where it helped.

In September 2023, the controller gained a custom iPhone storyboard, text commands, lidar visualization, speech-recognition work, system-volume tests, TCP no-delay tuning, layout cleanup, and an early Watch app. These changes are recognizably exploratory. UI, network behavior, and speech lifecycle were being discovered together.

What Codex should learn from this era

Do not begin a legacy-robot task by rewriting the language mix. First ask Codex to map ownership and data flow:

```

1 Inspect ROBController without changing files. Trace a command from the iPhone
2 or Watch UI through AutoNet serialization to the network send. Identify the
3 Objective-C/Swift boundary, thread or queue transitions, and every place a
4 stale command can survive. Cite files and line numbers. Mark uncertainty.

```

The useful output is a map, not code. Once the map is reviewed, request the smallest change that proves the model correct. This protects working legacy behavior while Codex builds context.

3. Era two: the solo workshop, August–September 2025

Cerebro's fresh repository begins August 5, 2025, but its initial import is already a substantial robot program. It includes a macOS storyboard, camera management, speech, serial control, AutoNet, lidar, Kinect/RealSense-era components, Leap Motion, RTSP video, task launchers, Core ML assets, and a large body of mixed Objective-C++ and Swift.

The following weeks read like a maker's engineering diary:

- Gemini is connected, speech delays are adjusted, and OAK camera input begins.
- Camera devices become discoverable at runtime.
- A new upper-neck servo is introduced and calibrated.
- RPLidar launch paths and wake-word handling are repaired.
- Output language propagates from controller to robot voice.
- Amber arm v1/v2 scripts are integrated, corrected, and exercised against real L10 and R11 arms.
- Keyframes and dual-arm sequences grow from manual experiments.
- Vision person segmentation, 3D body pose, SceneKit skeletons, and whole-body overlays are added.
- Speech wordiness, “shut up,” camera toggling, and head tracking become operator features.

The commit messages are valuable technical evidence. One warns that arm values are “very scary if the values go wrong.” Another records a damaged CAN connector. Another checkpoints code before rebuilding a pose view on every camera frame. These are not embarrassing details. They express the central truth of robotics: hardware provides the final test, and the cost of a bad assumption can be physical.

Turning workshop knowledge into specifications

Before asking Codex to touch hardware code, translate lived knowledge into invariants:

```

1 Goal: add a head-tilt command.
2
3 Constraints:
4 - Never emit a value outside the calibrated channel-2 range.
5 - A missing device must disable only head tilt.
6 - Manual stop must preempt queued motion.
7 - Preserve existing torso and camera behavior.
8 - Add a pure serialization fixture before editing the serial path.
9 - Do not claim hardware validation; give me an exact on-robot checklist.
```

This prompt is good because it tells Codex what must remain true. It does not dictate an implementation before the repository has been inspected.

4. The acceleration hinge: August 1, 2026

The paired August 1 commits transform both applications.

In ROBController, the large change introduces secure v2 transport, the Watch companion path, action and Watch command protocols, approval/status surfaces, autonomy controls, tests, and documentation.

In Cerebro, the corresponding change introduces the server side of the secure control plane, Gemini Robotics Live integration, an autonomy coordinator, camera H.264 transport for Vision Pro, a supervised Python runtime, system dependency management, action/video protocols, extensive tests, and a major documentation set.

This is the maturity jump. The old system primarily moved bytes and trusted its environment. The new system names roles, versions messages, authenticates peers, bounds queues, records operator intent, and fails closed.

Why this resembles AI-assisted engineering

No single clue proves AI use. Together, these clues make acceleration a reasonable inference:

1. Change size increases by an order of magnitude.
2. Both sides of a distributed protocol evolve together.
3. Production code arrives with fixture and integration tests.
4. Threat-model concerns—pinning, replay resistance, downgrade prevention, revocation, role checks—are addressed systematically.
5. Operator documentation explains contracts and incomplete capabilities.
6. Later commits use structured conventional-commit language and detailed implementation summaries.

The right conclusion is not “AI replaced the programmer.” It is “the builder acquired a fast engineering collaborator, and the repository began to preserve more of the reasoning that previously lived only in the builder’s head.”

5. Rebuilding the secure control plane with Codex

The v2 control plane is the strongest example of how to use Codex on a cross-repository change.

5.1 Start with a contract

Ask for a design before edits:

```
1 Analyze R0BController and Cerebro together. Design a versioned replacement for
2 the plaintext robot-control path. Requirements: QUIC with TLS 1.3, exact server
3 certificate pinning, per-device pairing secrets in Keychain, reciprocal proof
4 of possession, controller and lidar roles, persistent revocation, no automatic
5 legacy downgrade, bounded framing, and deterministic fixture tests. Identify
6 the files on both sides and list migration risks. Do not edit yet.
```

Review the design for four separations:

- **Identity:** which Cerebro instance is this?
- **Pairing:** has this device been enrolled?
- **Authorization:** is it an operator or telemetry publisher?
- **Freshness:** is this proof from the current connection rather than a replay?

Only then authorize implementation in checkpoints.

5.2 Implement the shared wire format first

Message formats should have explicit magic bytes, versions, lengths, limits, and error behavior. Generate the same golden fixtures on client and server. Codex is especially useful here because it can compare two implementations and find drift.

```

1 Implement only the v2 frame codec and fixtures in both repositories. Do not
2 open sockets and do not modify UI. Reject unknown mandatory versions, oversized
3 payloads, truncated frames, and invalid role/action combinations. Run the
4 fixture tests and show the exact diff.

```

5.3 Add transport without weakening authentication

QUIC/TLS encrypts a connection, but encryption alone does not tell a controller that it reached *its* robot. ROBController pins Cerebro's certificate. Pairing then proves possession of a separate per-device secret, while Cerebro resolves the device role from its own registry.

The August 10 fixes demonstrate why network handshakes need state-machine tests. iOS did not always expose Bonjour TXT metadata, and both peers could wait for the other to speak. The repair adds a padded pairing hello before the server sends its challenge, validates the controller ID, and preserves pinning and reciprocal proof.

A productive debugging prompt is:

```

1 Diagnose the QUIC pairing stall using both repositories. Draw the client and
2 server state machines from connection start through authenticated readiness.
3 Find any state where both peers await input. Preserve certificate pinning,
4 device-role lookup, and reciprocal proof. Add a regression fixture for the
5 deadlock before changing production code.

```

Never “fix” a handshake by skipping verification or silently falling back to plaintext. Availability bugs must not become authentication bugs.

6. Operator authority, Watch control, and autonomy

ROBController becomes more than a joystick in the mature system. It relays Watch intent, advertises accepted action capabilities, approves proposed robot actions, and starts a bounded autonomy session.

The autonomy model is intentionally a session, not indefinite permission. The `social_roam` profile captures an activation pose, stays within a radius, uses fresh lidar, moves slowly, and ends on manual control, explicit stop, or Cerebro restart. Meanwhile `ROBSerialBox` expires controller snapshots after three missed 5 Hz updates, sends one neutral/braked frame, then stops USB writes so the Arduino deadman can de-energize the treads.

This yields a useful authority hierarchy:

```

1 emergency / stop intent
2   > direct operator control
3   > bounded approved autonomy
4   > AI proposal
5   > idle

```

Ask Codex to encode this hierarchy in tests, not just comments:

```

1 Add table-driven tests for authority arbitration. Stop must preempt every
2 state. Manual control must end autonomy. Restart must default to autonomy off.
3 An expired controller snapshot must produce exactly one neutral/braked write
4 and no continuing heartbeat. Do not add arm movement.

```

The final sentence matters. Mature agents are capable of expanding a coherent feature beyond its safe scope. Tell Codex which adjacent capability is explicitly excluded.

7. Vision, depth, and video as bounded pipelines

Cerebro's camera path evolves from selecting webcams to a multi-provider perception system.

The 2026 implementation runs DepthAI in a supervised Python helper and sends synchronized RGB and aligned depth over a local user-only Unix socket. Keeping the SDK outside Cerebro isolates USB disconnects, malformed packets, missing packages, and Python crashes. AVFoundation remains an RGB fallback, but only one provider may own the OAK device.

The separate Vision Pro service uses its own authenticated QUIC connection, H.264 framing, bounded send state, and newest-frame policy. Slow video cannot back up robot control. Dropped frames trigger keyframe recovery rather than an unbounded queue.

The reusable design lesson is **separate by failure domain**:

- control and video use different services and queues;
- Python SDK faults do not terminate Cerebro;
- camera ownership is exclusive;
- perception drops old work instead of accumulating latency;
- an unavailable optional device degrades one capability.

Codex prompt:

```
1 Trace camera frames from provider to Vision, Gemini, and Vision Pro. For each
2 consumer record queue, ownership, backpressure, timestamp, and failure policy.
3 Then add the smallest change that keeps only the newest pending frame for the
4 slow consumer. Prove robot-control queues are untouched. Add malformed IPC and
5 slow-consumer tests; separate simulator evidence from hardware validation.
```

8. AI inside Cerebro: proposals, tools, and speech

The repositories currently document Gemini Robotics Live as the runtime model integration. OpenAI enters this story in two different ways:

1. **Codex develops and maintains the codebase.**
2. **The OpenAI Responses API can power a future model-neutral robot brain.**

These roles should not be conflated. A coding agent may edit a protocol while an application model later uses that protocol as a tool.

For an OpenAI-backed adapter, preserve Cerebro's deterministic boundaries. The model receives observations and returns structured proposals. Cerebro validates the schema, allow-list, session, deadline, and current authority. ROBController approves where required. Only a deterministic executor may touch hardware.

Conceptual Swift-like pseudocode:

```

1 let proposal = try await openAI.responses.create(
2   model: configuredModel,
3   input: currentRobotContext,
4   tools: [robotActionProposalSchema]
5 )
6
7 let action = try RobotActionProtocol.validate(proposal)
8 try authority.requireActiveSession(action.sessionID)
9 try policy.requireAllowed(action)
10 await operatorConsole.requestApproval(action)

```

Use the Responses API for multi-turn reasoning and tool calling, and choose the model through configuration rather than scattering a model name through UI and network code. Keep credentials outside source control. Do not send secrets, pairing material, faces, audio, or private surroundings unless the product's data policy and operator consent explicitly allow it.

A safe OpenAI integration task for Codex

```

1 Add an OpenAI Responses API provider behind the existing model-neutral
2 improvisation/proposal interface. First inspect the current official OpenAI
3 documentation. Keep the existing provider as default. Read the API key from a
4 secure runtime source, never source or logs. Return only the existing strict
5 schema. No model output may directly invoke serial, shell, network-control, or
6 arm code. Add mocked success, timeout, malformed output, refusal, and
7 cancellation tests. Do not enable the provider automatically.

```

The official OpenAI model guidance recommends the Responses API for reasoning, tool use, and multi-turn workflows. Models and capabilities change; let Codex consult current official documentation during implementation instead of freezing this book's August 2026 model names into the architecture.

9. Local improvisation and stage shows

The August 3 Cerebro change adds a safe local-improvisation layer, a connection-tolerant show coordinator, diagnostics, a llama.cpp provider, strict show schemas, fixture tests, and a Maker Faire opening show.

This feature demonstrates layered trust:

- authored dialogue is the mandatory fallback;
- local generation may select only allow-listed beats and delivery;
- Gemini may add live context to a trusted brief;
- show files cannot contain raw joints, servo values, hosts, ports, or shell;
- uncalibrated gestures fail closed;
- expired stage contexts permanently reject non-stop tool calls;
- `stop_motion` remains available through a priority stop lane.

Generative language becomes safe not because the model is guaranteed to behave, but because untrusted output has a narrow grammar and a deterministic fallback.

When asking Codex to extend a show format, request the validator and negative fixtures before the runner:

```

1 Extend robshow v1 with an optional audience-response beat. The JSON may select
2 only a named response policy and timeout. It may not contain executable code,
3 raw motor values, file paths, hosts, ports, or arbitrary tool names. Update the
4 schema, parser, negative fixtures, dry-run renderer, and docs before enabling
5 runtime behavior. Preserve authored fallback and stop preemption.

```

10. Dependencies are part of robot safety

Earlier Cerebro code contained machine-specific paths and assumed tools were installed. The mature version treats dependencies as fallible inputs.

Python can be selected, auto-detected, or created in an app-managed virtual environment. Package installation occurs only after an operator action. `sshpass`, `ticcmd`, and `RPLidar` are resolved and validated before launch. Missing optional tools disable their feature rather than crash Cerebro. Privileged MacPorts installation is shown to the operator in Terminal; the app does not capture an administrator password. The SSH password travels through an anonymous pipe rather than the process argument list.

This is an excellent Codex refactoring pattern:

```

1 Inventory every Process/NSTask launch in Cerebro. Do not edit yet. Classify the
2 executable as required or optional; list path source, validation, credentials,
3 timeout, restart behavior, and user-visible failure. Then centralize only the
4 shared preflight while preserving each feature's behavior. Add tests for
5 missing, directory, non-executable, and disappearing executables.

```

Codex can search every launch site more reliably than a tired human, but the operator must decide which dependency failures are tolerable.

11. A repeatable Codex workflow for these repositories

Step 1: protect the worktree

Start every session with:

```

1 Inspect git status in ROBController and Cerebro. Existing changes belong to me.
2 Do not overwrite, revert, stage, or commit them. Identify generated Xcode user
3 state separately. Summarize the relevant architecture before editing.

```

The current worktrees, for example, contain modified `UserInterfaceState.xcuserstate` files. They are user state, not permission to discard anything.

Step 2: state outcome and invariants

Prefer “When control packets stop, the base brakes once and becomes silent” to “refactor ROBSerialBox.” The former is testable and safety-relevant.

Step 3: ask for read-only diagnosis

For cross-language and cross-repository behavior, have Codex trace the entire path before implementation. Ask it to cite evidence and say what it cannot know without hardware.

Step 4: divide at stable boundaries

Good checkpoints are codec, fixtures, client state machine, server state machine, UI, and documentation. Avoid one request that changes wire format, physical actuation, and user experience without intermediate review.

Step 5: make Codex edit and verify

```
1 Implement the approved codec checkpoint. Preserve unrelated changes. Run the
2 narrow fixture tests first, then the relevant build. Review your own diff for
3 security, concurrency, compatibility, and accidental secrets. Report commands,
4 results, remaining hardware checks, and files changed.
```

Step 6: inspect the diff yourself

Use `git diff --check`, `git diff --stat`, and `git diff`. Pay special attention to entitlements, Info.plist permissions, Xcode project membership, networking, Keychain accessibility, dispatch queues, and motor values.

Step 7: validate in layers

1. Pure codec/schema fixtures
2. Unit tests with mocked time and I/O
3. Client/server integration tests on loopback
4. Xcode build and static diagnostics
5. Simulator/UI checks where meaningful
6. Bench hardware with motors disabled or lifted
7. Low-speed supervised robot test with an accessible stop

Codex may complete layers one through five. It must never imply that they prove layers six and seven.

Step 8: commit the reasoning

A mature message explains behavior and risk:

```
1 fix(network): prevent QUIC pairing handshake deadlock
2
3 - send a bounded client hello before awaiting the challenge
4 - validate device identity against the paired registry
5 - preserve certificate pinning and reciprocal proof
6 - add a regression fixture for simultaneous-read startup
```

The history then becomes usable context for the next human or AI session.

12. Prompt recipes for the actual change families

Cross-repository protocol change

- 1 Trace protocol X in both repos. Produce a compatibility matrix for old client,
- 2 new client, old server, and new server. Design golden fixtures before edits.
- 3 Reject malformed and oversized input. No automatic security downgrade. Make
- 4 changes in reviewable checkpoints and run both test suites.

Concurrency bug

- 1 Build a state machine from code, queues, callbacks, and cancellation paths.
- 2 Identify races, double completion, and mutual waits. Add a deterministic
- 3 regression test. Keep UI updates on the main actor/queue and network state on
- 4 its owning queue. Do not mask the bug with sleeps.

Camera crash

- 1 Diagnose only first. Trace device discovery, selected device lifetime, session
- 2 mutation, and frame consumers. Explain the exact crash precondition. Separate
- 3 what can be fixture-tested from the rapid-toggle test required on the robot Mac.

New AI tool

- 1 Treat model output as untrusted. Define a versioned strict schema, limits,
- 2 allow-list, authority check, deadline, cancellation, idempotency rule, operator
- 3 approval, terminal result, and audit event. Add negative fixtures. Do not wire
- 4 physical execution until the protocol review is complete.

Arm feature

- 1 No motion yet. Inventory joint names, signs, units, calibrated ranges, feedback,
- 2 URDF assumptions, transforms, collision checks, and emergency stop path. Mark
- 3 every missing fact. Build a dry-run command preview and fixtures. Require a
- 4 separate explicit request before enabling hardware output.

Documentation from code

- 1 Update the operator document from the implemented diff. Distinguish implemented,
- 2 tested in fixtures, built, simulator-tested, and hardware-validated. List known
- 3 gaps plainly. Do not describe planned behavior as present behavior.

13. Reviewing AI-generated robot code

An AI-generated diff is a proposal. Review it in five passes.

Safety

- Does loss of input converge to neutral?
- Is stop independent, prioritized, and idempotent?

- Are speeds, positions, payload sizes, and durations bounded?
- Does restart return to a safe state?
- Does an unavailable sensor prevent motion that depends on it?

Security

- Are identity, enrollment, authorization, and freshness distinct?
- Are secrets absent from source, logs, process arguments, and Bonjour?
- Is there any fallback to plaintext or unauthenticated behavior?
- Can revocation terminate a live session?
- Can a telemetry role become a control role by editing its payload?

Concurrency

- Which queue owns each mutable state variable?
- Can both peers wait forever?
- Can cancellation race with completion or a late tool call?
- Can a slow camera/model consumer block control?

Compatibility

- Are protocol versions explicit?
- Are unknown fields tolerated only where intended?
- Do both repositories share golden fixtures?
- Is legacy behavior opt-in, observable, and removable?

Evidence

- What tests ran?
- What could not run in the environment?
- Was hardware actually exercised?
- Are documentation claims no stronger than the evidence?

14. What remains intentionally unfinished

Mature engineering is visible in the features a system refuses to fake. Cerebro reports general picking and arm motion as unavailable because the repository still lacks calibrated camera-to-arm transforms, complete neck mapping, joint feedback, inverse kinematics, collision checking, and a verified grasp executor. Gesture cues fail closed without a calibrated catalog and feedback-capable executor. Approval records operator intent but does not by itself start physical action.

These are the next safe milestones:

1. Establish measured coordinate frames and calibration artifacts.
2. Add feedback and freshness to joint state.
3. Build offline IK with joint and velocity bounds.
4. Add collision models and swept-volume checks.
5. Simulate and dry-run named motions.
6. Validate one arm on a bench at low speed.
7. Introduce a deterministic executor with stop preemption.
8. Only then expose a narrow, schema-constrained AI proposal tool.

Codex can help at every milestone, but it cannot supply missing physical facts from code. Calibration measurements, payload limits, wiring, clearances, and the location of humans around the robot belong to the real world.

Epilogue: authorship after acceleration

The early repositories preserve the voice of one person working from several machines: informal, immediate, hardware-aware, and willing to checkpoint an uncertain experiment. The newer code does not erase that voice. It gives it leverage.

The mature system is better not merely because it contains more code. It has contracts where there were assumptions, bounded queues where there could be backlogs, explicit authority where there was trust, tests where there was only memory, and documentation where knowledge once lived in the workshop.

That is the best use of Codex in robotics. Let AI search widely, trace deeply, draft quickly, compare both sides of a protocol, and remember every edge case you name. Keep the human responsible for scope, evidence, physical validation, and the decision to energize the machine.

Sources and further reading

Repository sources

- ROBController Git history through eb57375, August 10, 2026
- Cerebro Git history through d45f9c9, August 10, 2026
- ROBController/docs/rob-control-v2-and-autonomy.md
- ROBController/docs/robot-action-console.md
- ROBController/docs/watch-controller.md
- Cerebro/docs/rob-control-v2.md
- Cerebro/docs/controller-activated-autonomy.md
- Cerebro/docs/depth-camera.md
- Cerebro/docs/vision-pro-video.md

- [Cerebro/docs/gemini-robotics-live.md](#)
- [Cerebro/docs/gemini-robotics-stage-action-plan.md](#)
- [Cerebro/docs/local-improvisation-provider.md](#)

Official OpenAI documentation

- [Codex use cases](#)¹
- [OpenAI model guidance](#)²
- [OpenAI API models](#)³

Consult the current official documentation when implementing; models, capabilities, availability, and SDK shapes can change after this edition.

¹<https://developers.openai.com/codex/use-cases>

²<https://developers.openai.com/api/docs/guides/latest-model>

³<https://developers.openai.com/api/docs/models>